

05 - Greedy Algorithms

Algorithms & Complexity

Eduard Torres Montiel - eduard.torres@udl.cat

Logic & Optimization Group, University of Lleida

17/04/2024

- 1 Introduction
- 2 Example 1: Scheduling Activities
- 3 Example 2: The Coin Change Problem
- 4 Bibliography

Introduction

Which is the difference between these set of problems:

- Sorting an array of integers in descending order.
- Search an element in an array.
- Solving a 9x9 Sudoku.
- Solving the n -queens puzzle.

And these others:

- Compute a car journey between Igualada and Lleida.
- Assign mutually-exclusive tasks to a processor to minimize its idle time.
- Given an initial configuration of n queens, solve the n -queens puzzle *minimizing the number of moves*.

Introduction

Which is the difference between these set of problems:

- Sorting an array of integers in descending order.
- Search an element in an array.
- Solving a 9x9 Sudoku.
- Solving the n -queens puzzle.

And these others:

- Compute a car journey between Igualada and Lleida.
- Assign mutually-exclusive tasks to a processor to minimize its idle time.
- Given an initial configuration of n queens, solve the n -queens puzzle *minimizing the number of moves*.

The first ones only have *one possible solution* (or several, but all of them are *equivalent*).

The second ones have *many possible solutions*. Each solution has an associated **value** that we want to **minimize** or **maximize**. These problems are called **optimization problems**.

Optimization problems

An **optimization problem** is a kind of problem that can have *many possible solutions*.

Each solution has a **value**, and we want to find a solution with the **optimal** (minimum or maximum) value.

We call such a solution **an optimal solution**, as opposed to *the optimal solution*, as there might be several solutions that achieve the optimal value.

For most of the optimization problems, finding *an optimal solution* is a challenging task, especially as the size of the *instance* grows.

To start, we will study the **greedy** algorithmic approach, which for some problems can find an *optimal* solution, but for many others will find *suboptimal* solutions using a *reasonable* amount of resources.

Example 1: Scheduling Activities

Consider the problem of scheduling several competing activities that require exclusive use of a common resource. Our goal is to **select a maximum-size set of mutually compatible activities**.

We are presented with a set $S = \{a_1, a_2, \dots, a_n\}$ of n activities.

Each activity a_i has a **start time** s_i and a **finish time** f_i , where $0 \leq s_i < f_i < \infty$.

If selected, activity a_i takes place during the half-open time interval $[s_i, f_i)$.

Activities a_i and a_j are compatible if their intervals do not overlap: $s_i \geq f_j$ or $s_j \geq f_i$.

Assume that the activities are sorted according to their *finish time*:

$$f_1 \leq f_2 \leq f_3 \leq \dots \leq f_{n-1} \leq f_n$$

Example of activities with their *start* and *finish* time:

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	7	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

A subset of mutually compatible activities?

Example of activities with their *start* and *finish* time:

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	7	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

A subset of mutually compatible activities?

■ $\{a_3, a_9, a_{11}\}$.

Is this a *maximum subset*?

Example of activities with their *start* and *finish* time:

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	7	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

A subset of mutually compatible activities?

- $\{a_3, a_9, a_{11}\}$.

Is this a *maximum subset*?

- **No**, the subset $\{a_1, a_4, a_8, a_{11}\}$ is larger. In fact, it is a *largest subset of mutually compatible activities*. Another one is $\{a_2, a_4, a_9, a_{11}\}$.

Solving the problem

How can we solve this problem? Let's analyze the following observation:

Either you take activity 1 or you don't.

Let B denote the set of activities that *end before* activity 1 starts, and A the set of activities that *start after* activity 1 ends:

$$B = \{i | 2 \leq i \leq n \text{ and } f_i < s_1\}$$

$$A = \{i | 2 \leq i \leq n \text{ and } s_i > f_1\}$$

If activity 1 is in the optimal schedule, then so are the optimal schedules for B and A (which we can find recursively).

If not, we can find the optimal schedule for $\{2, 3, \dots, n\}$ recursively.

So we should try both choices and take whichever one gives us the better schedule!

This approach could be solved using **dynamic programming**, a method that we will study at the end of the subject.

However, we will not need to apply that! Consider the following **greedy** observation:

We'd like the first activity to finish as early as possible, so this leaves us with the largest number of remaining activities.

This intuition suggests the following simple *greedy algorithm*:

- 1 Scan through the classes in order of finish time.
- 2 Whenever you encounter a class that doesn't conflict with your latest class so far, take it!

Pseudocode

Algorithm 1 The Greedy Activity Selector algorithm.

Precondition: $F[1 : n]$ an array with the finish times of n activities sorted ascending and $S[1 : n]$ an array with their start times.

Postcondition: A a subset of mutually compatible activities.

procedure GREEDYACTIVITYSELECTOR($S[1 : n], F[1 : n]$)

$A \leftarrow \{a_1\}$

$k \leftarrow 1$

for $m \leftarrow 2$ to n **do**

if $S[m] \geq F[k]$ **then**

$A \leftarrow A \cup \{a_m\}$

$k \leftarrow m$

return A

Complexity of the algorithm?

Pseudocode

Algorithm 2 The Greedy Activity Selector algorithm.

Precondition: $F[1 : n]$ an array with the finish times of n activities sorted ascending and $S[1 : n]$ an array with their start times.

Postcondition: A a subset of mutually compatible activities.

procedure GREEDYACTIVITYSELECTOR($S[1 : n], F[1 : n]$)

$A \leftarrow \{a_1\}$

$k \leftarrow 1$

for $m \leftarrow 2$ to n **do**

if $S[m] \geq F[k]$ **then**

$A \leftarrow A \cup \{a_m\}$

$k \leftarrow m$

return A

Complexity of the algorithm?

If $F[1 : n]$ is already sorted then $O(n)$.

Is this an optimal solution?

To prove that our algorithm yields an *optimal solution* we need to use a little bit of induction, starting with the *base case*:

- Let f be the activity that finishes first.
- Suppose we have a maximal conflict-free schedule X that does not include f .
- Let g be the first activity in X to finish.
- Since f finishes before g does (remember f is the activity that finishes *first*), f cannot conflict with any activity in the set $X \setminus \{g\}$.
- Thus, the schedule $X' = X \cup \{f\} \setminus \{g\}$ is also conflict-free.
- Since X' has the same size as X , it is also maximal.

To finish the proof we would have to develop the general induction case (which can be derived easily from the base case).

Example 2: The Coin Change Problem

Consider an array C that contains the value (in cents) of the different euro coins:

$$C = [200, 100, 50, 20, 10, 5, 2, 1]$$

Consider that we have an *unlimited* amount of each coin. The goal is to use the *minimum number of coins* to return the change v to a client.

Example 2: The Coin Change Problem

Consider an array C that contains the value (in cents) of the different euro coins:

$$C = [200, 100, 50, 20, 10, 5, 2, 1]$$

Consider that we have an *unlimited* amount of each coin. The goal is to use the *minimum number of coins* to return the change v to a client.

Example: Which coins can we use to reach €5.27?

Example 2: The Coin Change Problem

Consider an array C that contains the value (in cents) of the different euro coins:

$$C = [200, 100, 50, 20, 10, 5, 2, 1]$$

Consider that we have an *unlimited* amount of each coin. The goal is to use the *minimum number of coins* to return the change v to a client.

Example: Which coins can we use to reach €5.27?

- 5 of €1, 1 of 20c, 1 of 5c and 1 of 2c (8 coins). Is this *the minimum number of coins that we can use*?

Example 2: The Coin Change Problem

Consider an array C that contains the value (in cents) of the different euro coins:

$$C = [200, 100, 50, 20, 10, 5, 2, 1]$$

Consider that we have an *unlimited* amount of each coin. The goal is to use the *minimum number of coins* to return the change v to a client.

Example: Which coins can we use to reach €5.27?

- 5 of €1, 1 of 20c, 1 of 5c and 1 of 2c (8 coins). Is this *the minimum number of coins that we can use*?
- **No**, we can use less coins by selecting 2 of €2, 1 of €1, 1 of 20c, 1 of 5c and 1 of 2c (6 coins). In fact, this is *an optimal solution*.

In this problem we follow the same principle than in the *Scheduling Activities* problem: we either use the coin $T[i]$ or not.

As in the previous problem, we need to sort the array C , in this case in decreasing order.

Then, we consider the following **greedy** observation:

We select as many of the largest coins as possible, as this leaves us with the smallest remainder of v to return.

This intuition suggests the following simple *greedy algorithm*:

- 1 Select as many coins with the largest value so far.
- 2 Subtract from v the value that we already covered with these coins.
- 3 Repeat with the next largest value.

Pseudocode

Algorithm 3 The Greedy Coin Change algorithm.

Precondition: v a value (in cents) that we have to return.

Postcondition: R an array with the number of coins of each type such that their value is equals to v .

```
procedure GREEDYCOINCHANGE( $v$ )  
   $C \leftarrow [200, 100, 50, 20, 10, 5, 2, 1]$   
   $R \leftarrow [0, 0, 0, 0, 0, 0, 0, 0]$   
   $i \leftarrow 1$   
  while  $v > 0$  and  $i \leq \text{len}(C)$  do  
     $R[i] \leftarrow v / C[i]$   
     $v \leftarrow v \% C[i]$   
     $i \leftarrow i + 1$   
  return  $R$ 
```

Pseudocode

Algorithm 4 The Greedy Coin Change algorithm.

Precondition: v a value (in cents) that we have to return.

Postcondition: R an array with the number of coins of each type such that their value is equals to v .

```
procedure GREEDYCOINCHANGE( $v$ )  
   $C \leftarrow [200, 100, 50, 20, 10, 5, 2, 1]$   
   $R \leftarrow [0, 0, 0, 0, 0, 0, 0, 0]$   
   $i \leftarrow 1$   
  while  $v > 0$  and  $i \leq \text{len}(C)$  do  
     $R[i] \leftarrow v / C[i]$   
     $v \leftarrow v \% C[i]$   
     $i \leftarrow i + 1$   
  return  $R$ 
```

Best case and worst case? Complexity?

Pseudocode

Algorithm 5 The Greedy Coin Change algorithm.

Precondition: v a value (in cents) that we have to return.

Postcondition: R an array with the number of coins of each type such that their value is equals to v .

```
procedure GREEDYCOINCHANGE( $v$ )  
   $C \leftarrow [200, 100, 50, 20, 10, 5, 2, 1]$   
   $R \leftarrow [0, 0, 0, 0, 0, 0, 0, 0]$   
   $i \leftarrow 1$   
  while  $v > 0$  and  $i \leq \text{len}(C)$  do  
     $R[i] \leftarrow v / C[i]$   
     $v \leftarrow v \% C[i]$   
     $i \leftarrow i + 1$   
  return  $R$ 
```

Best case and worst case? Complexity?

- *Best case:* $v \% 200 = 0$. $O(1)$.
- *Worst case:* $v \% 1 = 0$. $O(n)$, if we consider that we have n different coins.

Is this an optimal solution?

As we did in the previous example, we need to use a little bit of *induction*, starting with the *base case*:

- Consider there is an *optimal solution* that does not include the largest coin. We have two possible scenarios:
 - $v < C[1]$. We will never be able to include this coin to the solution, the *optimal solutions* do not include $C[1]$.
 - $v \geq C[1]$. Then we could at least include one copy of this coin. This means we included more coins than the required, as the value of the next coins are $< C[1]$. Therefore there is an *optimal solution* that includes one or more copies of $C[1]$.

Again, the inductive step is easily derived from the base case.

Bibliography

- T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein. *Introduction to Algorithms, 4th Edition*. MIT Press. Chapter 15.
- Jeff Erickson. *Algorithms*.
<http://jeffe.cs.illinois.edu/teaching/algorithms/>. Chapter 4.